TRAINING & REGULARIZATION

ThS. Lê Thị Thùy Trang

2026-04-06

1 Giới thiệu bài thực hành

1.1 Mục tiêu bài thực hành

Sau bài lab này, sinh viên xây dựng một pipeline huấn luyện hoàn chỉnh cho bài toán phân loại ảnh lỗi bề mặt thép bằng mạng nơ-ron truyền thẳng (MLP). Mục tiêu chính của buổi học không phải là chạy thật nhiều mô hình để đạt điểm số cao nhất, mà là hiểu đúng quy trình huấn luyện một mô hình học sâu

- Chuẩn bị dữ liệu đúng cách;
- Chọn hàm mất mát phù hợp;
- Hiểu vai trò của train/ validation/ test;
- Theo dõi hiện tượng overfitting và underfitting;
- So sánh optimizer;
- áp dụng regularization;
- Ghi lại toàn bộ quá trình bằng `Weights & Biases`

1.2 Kết quả đầu ra mong đợi của bài lab

Sau bài lab này, sinh viên cần có:

1. Một repo chạy được;
2. Một baseline MLP cho bài toán phân loại 6 lớp;
3. Ít nhất 3 run khác nhau trên `W&B`;
4. Learning curves cho train và validation;
5. Một bảng so sánh các cấu hình;
6. Kết luận mô hình nào tốt nhất và vì sao;
7. Đánh giá cuối cùng trên test set.

1.3 Những thứ bắt buộc phải nộp

Mỗi sinh viên cần nộp tối thiểu:

1. Repo code chạy được.
2. README hướng dẫn chạy.

3. Link hoặc ảnh chụp `W&B` dashboard.
4. Bảng so sánh ít nhất 3 cấu hình.
5. Learning curves.
6. Kết quả test của best model.
7. Phân tích ngắn về overfitting / underfitting.
8. Mô tả kiến trúc mạng, cấu hình đã thử nghiệm và kết luận cấu hình nào tốt nhất và vì sao.

1.4 Bối cảnh bài toán

Chúng ta đang đóng vai nhóm kỹ sư AI hỗ trợ bộ phận QA trong dây chuyền sản xuất thép. Mỗi ảnh đầu vào là một ảnh grayscale chụp bề mặt thép. Mô hình cần dự đoán ảnh đó thuộc loại lỗi nào.

Bộ dữ liệu sử dụng là **NEU Surface Defect Database**, sinh viên truy cập đường dẫn này để lấy dữ liệu. Tập dữ liệu gồm 6 lớp, tương ứng với các loại lỗi:

- Crazing
- Inclusion
- Patches
- Pitted Surface
- Rolled-in Scale
- Scratches

Pipeline bài toán dừng ở mức huấn luyện một mạng nơ-ron MLP để sinh viên hiểu các kỹ thuật huấn luyện và tối ưu hoá mô hình học sâu:

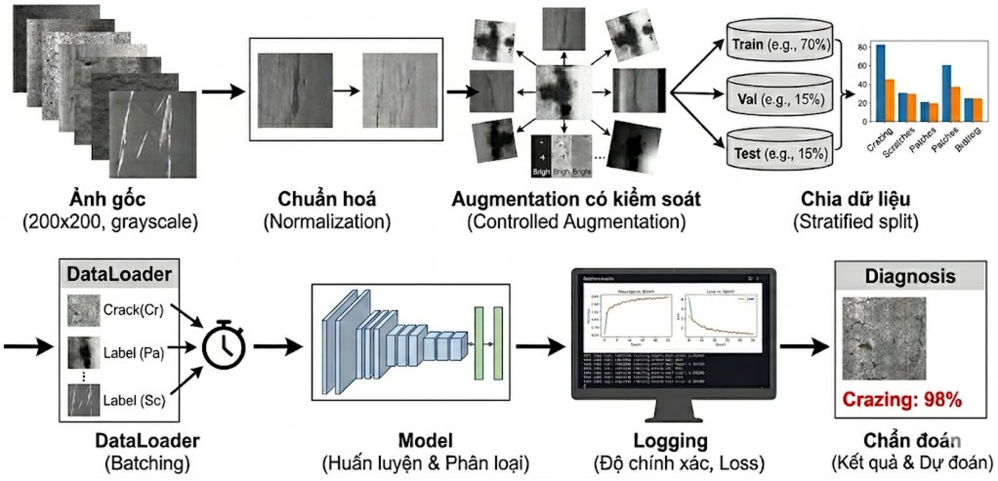

Figure 1: Pipeline project

1.5 Cấu trúc repo starter kit

Repo của bài lab có cấu trúc như sau:

Repo starter kit hiện tại có cấu trúc như sau:

```
csc4005_lab1_neu_mlp_starter/
 README.md
 REPORT_TEMPLATE.md
 requirements.txt
 configs/
    baseline.json
 docs/
    WANDB_GUIDE.md
    LAB_GUIDE_LAB1.md
 notebooks/
    lab1_demo.ipynb
 outputs/
 src/
    __init__.py
    dataset.py
    model.py
    train.py
    utils.py
 ci/
     check_structure.py
     smoke_train.py
```

Ý nghĩa từng phần

- `src/dataset.py`: đọc dữ liệu, split train/val/test, tạo DataLoader.
- `src/model.py`: định nghĩa mô hình MLP.
- `src/train.py`: huấn luyện mô hình, đánh giá, log W&B.
- `src/utils.py`: các hàm tiện ích như vẽ curves, confusion matrix, lưu metrics.
- `docs/WANDB_GUIDE.md`: hướng dẫn riêng cho W&B.
- `REPORT_TEMPLATE.md`: mẫu báo cáo cho sinh viên.

2 Giới thiệu công cụ Weights & Bias

Weights & Biases (W&B) là một nền tảng giúp theo dõi, lưu trữ, so sánh và quản lý các thí nghiệm machine learning / deep learning.

Hãy tưởng tượng bạn chạy 3 mô hình: mô hình A dùng AdamW; mô hình B dùng SGD; mô hình C tăng dropout và weight decay.

Nếu chỉ nhìn log trong terminal, sau một lúc bạn sẽ quên: run nào dùng learning rate bao nhiêu? mô hình nào có val accuracy tốt nhất? val loss của run nào ổn định hơn? epoch nào là lúc mô hình tốt nhất? cấu hình nào gây overfitting mạnh hơn?

`W&B` giúp bạn tránh hoàn toàn kiểu làm việc ước chừng đó.

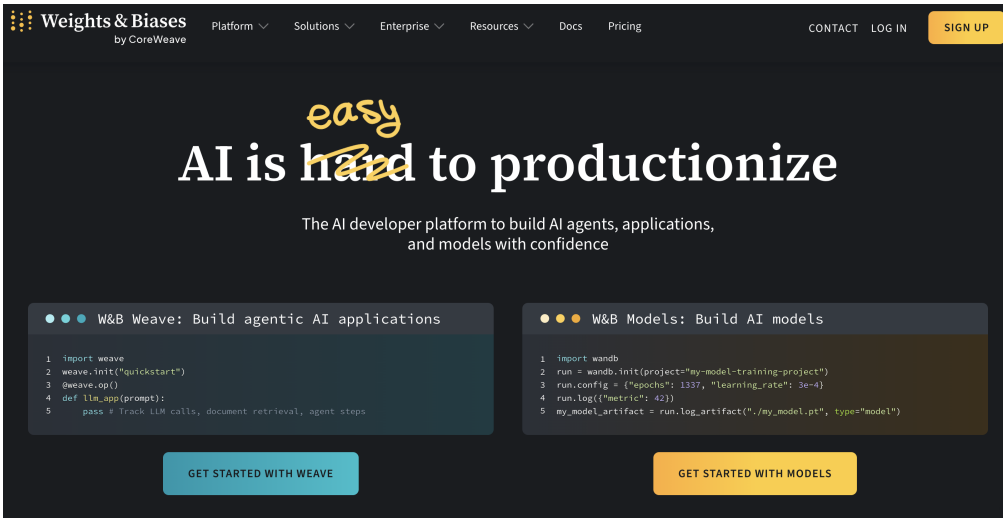

Figure 2: Weights & Biases

`W&B` cho phép:

- Lưu lại cấu hình của từng lần chạy;
- Ghi lại train loss, val loss, train accuracy, val accuracy theo từng epoch;
- So sánh nhiều run trên cùng một dashboard;
- Xem learning curve trực quan;
- Lưu best metrics;
- Hỗ trợ viết báo cáo vì mọi thứ đã có log rõ ràng;
- Giúp thí nghiệm có thể tái lập được.

2.1 Lý do lựa chọn công cụ Weights & Bias

Bài lab yêu cầu sinh viên:

1. So sánh ít nhất 3 cấu hình;
2. Có learning curves;
3. Có bảng tổng hợp configs -> metrics;
4. Chọn best config và giải thích vì sao.

Nếu không dùng `W&B`, sinh viên vẫn có thể chạy mô hình, nhưng sẽ rất khó quản lý kết quả một cách khoa học. Với `W&B`, sinh viên không chỉ chạy được mà còn chứng minh được kết luận của mình bằng dữ liệu thí nghiệm. Tóm lại, `W&B` không làm mô hình thông minh hơn, nhưng làm quá trình thực nghiệm chuyên nghiệp hơn.

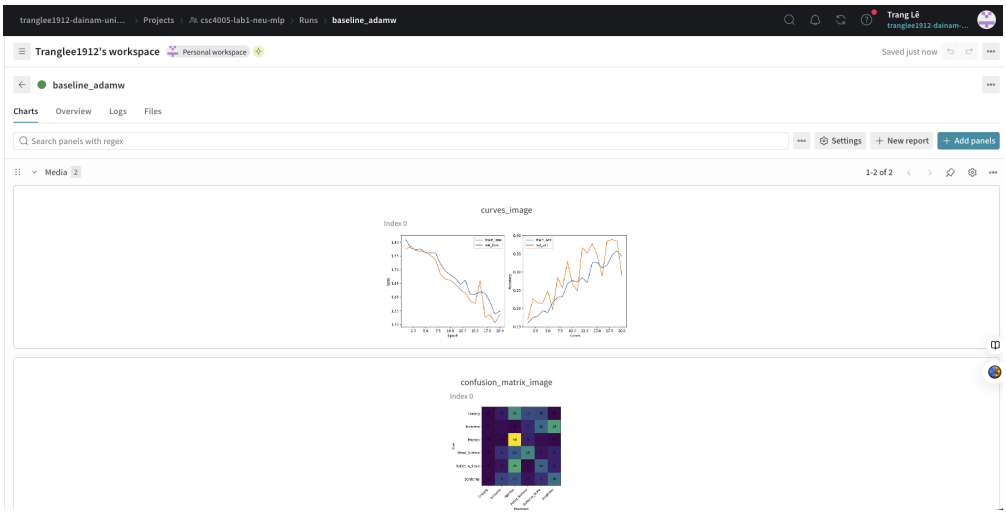

Figure 3: Log trên Weights & Biases

3 Chuẩn bị môi trường thực hành

3.1 Điều kiện cần

Sinh viên cần:

- Có tài khoản GitHub và đăng nhập được.
- Máy có Internet.
- Môi trường ảo `csc4005-dl` đã được set-up từ buổi học trước.

3.2 Clone repo

Bấm **Accept Assignment** theo link đã cung cấp trên `Notion`.

Clone repo về máy

```
git clone <repo-url>
cd <ten-repo>
```

3.3 Khởi tạo environment

```
conda activate csc4005-dl
```

3.4 Cài thư viện phục vụ bài lab

Về cơ bản, các thư viện đã cài đặt ở bài trước là đủ cho bài thực hành, sinh viên chỉ cần bổ sung thư viện trong file `requirements.txt`:

```
pip instal -r requirements.txt
```

Sau khi cài xong các thư viện trong `requirements.txt`, thực hiện cài đặt và sử dụng W&B

3.5 Cài đặt và sử dụng W&B

3.5.1 Tạo tài khoản

Truy cập trang W&B và đăng ký tài khoản. Bạn có thể dùng email cá nhân hoặc email trường.

3.5.2 Lấy API key

Sau khi đăng nhập, vào phần profile/settings của tài khoản để lấy **API key**.

3.5.3 Đăng nhập trên máy

Trong terminal, chạy:

```
wandb login
```

Dán API key khi được yêu cầu.

Nếu thành công, W&B sẽ báo rằng thiết bị của bạn đã được liên kết với tài khoản.

3.5.4 Chạy offline khi cần

Nếu không có internet hoặc mạng không ổn định, có thể dùng chế độ offline:

```
wandb offline
```

Khi có mạng lại, có thể đồng bộ sau. Tuy nhiên, trong lab này, sinh viên nên ưu tiên chạy online để dễ quan sát dashboard.

3.5.5 Kiểm tra nhanh W&B đã hoạt động chưa

Sau khi chạy một training run có bật `--use_wandb`, terminal thường sẽ in ra đường dẫn tới project trên W&B. Hãy mở link đó bằng trình duyệt. Nếu dashboard hiện ra run và metrics, bạn đã kết nối thành công.

4 Voila, hướng dẫn thực hành ở đây

4.1 Chạy baseline đầu tiên

Tinh thần của các bài lab do cô Trang Lê chắp bút là chạy đã rồi sửa đổi sau. Trước khi mạo hiểm thử nghiệm các model hot-trend, hãy bước những bước đi đầu tiên với một model MLP giản đơn và sử dụng công cụ `W&B` để huấn luyện model và theo dõi thí nghiệm một cách chuyên nghiệp.

Để dừng phần flex tại đây thì chúng ta chạy một baseline ổn định đã:

```
python -m src.train \
  --data_dir /duong_dan/NEU-CLS.zip \
  --project csc4005-lab1-neu-mlp \
  --run_name baseline_adamw \
  --optimizer adamw \
  --lr 0.001 \
  --weight_decay 0.0001 \
  --dropout 0.3 \
```

```
  --epochs 20 \
  --batch_size 32 \
  --img_size 64 \
  --patience 5 \
  --augment \
  --use_wandb
```

Nhiều tham số quá nhỉ, chịu khó đọc giải thích tham số nha

- `--data_dir`: **bắt buộc**, là đường dẫn tới `NEU-CLS.zip` hoặc thư mục dữ liệu.
- `--project`: tên project trên W&B. Trong lab này dùng thống nhất là `csc4005-lab1-neu-mlp`.
- `--run_name`: tên run.
- `--optimizer`: chọn optimizer, ví dụ `adamw` hoặc `sgd`.
- `--scheduler`: có thể là `none` hoặc `plateau`.
- `--lr`: learning rate.
- `--weight_decay`: hệ số regularization L2.
- `--dropout`: hệ số dropout.
- `--epochs`: số vòng lặp huấn luyện.
- `--batch_size`: số mẫu trong một batch.
- `--img_size`: kích thước ảnh sau resize.
- `--patience`: số epoch chờ để early stopping.
- `--hidden_dims`: số neuron trong các hidden layers.
- `--augment`: bật augmentation cho train set.
- `--use_wandb`: bật logging lên W&B.

Chạy baseline xong thì công việc vẫn chờ đợi các em phía trước, thực hiện tuần tự các bước sau:

1. Mở dashboard `W&B`.
2. Kiểm tra xem train loss và val loss có được log hay không.
3. Kiểm tra train accuracy và val accuracy.
4. Ghi chú lại kết quả baseline để làm mốc so sánh.

4.2 Nếu nhìn những biểu đồ như nhìn bức vách thì không sao hết, đọc hướng dẫn dưới đây là ổn áp ngay

Khi training xong, `W&B` sẽ cho chúng ta các đường cong học tập.

4.2.1 Những đường quan trọng nhất

- `train_loss`
- `val_loss`
- `train_acc`
- `val_acc`

4.2.2 Trường hợp tốt

- `train_loss` giảm đều,
- `val_loss` giảm đều hoặc giảm rồi ổn định,
- `train_acc` tăng,
- `val_acc` tăng theo.

Đây là dấu hiệu mô hình đang học đúng hướng.

4.2.3 Dấu hiệu overfitting

- `train_acc` tăng rất cao,
- `train_loss` tiếp tục giảm,
- nhưng `val_loss` bắt đầu tăng,
- `val_acc` không tăng tương ứng hoặc giảm.

Lúc này mô hình đang ghi nhớ train set quá mức.

4.2.4 Dấu hiệu underfitting

- cả train accuracy và val accuracy đều thấp,
- train loss và val loss đều còn cao.

Lúc này mô hình chưa học đủ hoặc mô hình quá yếu.

4.3 Chạy thêm ít nhất 2 cấu hình để so sánh

Lab yêu cầu **so sánh ít nhất 3 cấu hình**. Vì vậy, ngoài baseline, ta cần chạy thêm tối thiểu 2 run nữa. Với Run B và Run C có thể đổi optimizer hay tăng regularization gì đó tuỳ các em. Tuy nhiên, lưu ý cách làm thí nghiệm cho đúng.

Một lỗi rất hay gặp là thay đổi quá nhiều thứ cùng lúc, ví dụ:

- vừa đổi optimizer,
- vừa đổi batch size,
- vừa đổi hidden dims,
- vừa đổi dropout,
- vừa đổi learning rate.

Nếu làm như vậy, khi kết quả thay đổi bạn sẽ **không biết nguyên nhân thực sự đến từ đâu**.

Nguyên tắc nên áp dụng

- Muốn so sánh optimizer → giữ các yếu tố khác gần giống nhau.
- Muốn xem dropout có tác dụng gì → chỉ đổi dropout trước.
- Muốn đánh giá weight decay → chỉ đổi weight decay hoặc đổi rất ít yếu tố kèm theo.

Làm thí nghiệm có kiểm soát là điều rất quan trọng trong machine learning/ deep learning.

4.4 So sánh các run trên W&B

Sau khi chạy ít nhất 3 cấu hình, vào project trên W&B và so sánh.

- run nào có `best_val_acc` cao nhất,
- run nào có `val_loss` thấp nhất,
- run nào learning curve ổn định hơn,
- run nào ít overfitting hơn.

Lưu ý: Kết luận phải dựa trên số liệu và dashboard, không nên viết cảm tính.

4.5 Bước cuối cùng: so bó đũa, chọn cột cờ

4.5.1 Chọn best model

Best model được chọn dựa trên kết quả train model

Nguyên tắc đúng là:

1. So sánh các mô hình trên **validation set**.
2. Chọn **một cấu hình tốt nhất**.
3. Ghi rõ lý do chọn.
4. Chỉ sau đó mới dùng **test set** để đánh giá cuối cùng.

Tiêu chí chọn best config có thể là:

- val accuracy cao nhất,
- val loss thấp và ổn định,
- learning curves đẹp,
- ít overfitting hơn.

Hết sức lưu ý: Không được chọn mô hình chỉ vì train accuracy cao.

4.5.2 Đánh giá trên test set

Sau khi chọn được best config, sinh viên đánh giá trên test set **một lần cuối**.

Kết quả nên có:

- test accuracy,
- classification report,
- confusion matrix,
- một vài ví dụ dự đoán đúng,
- một vài ví dụ dự đoán sai.

Mỗi run sẽ được lưu trong:

```
outputs/<run_name>/
```

Thông thường sẽ có:

- `best_model.pt`

- `history.csv`
- `curves.png`
- `confusion_matrix.png`
- `metrics.json`

5 Những lỗi rất hay gặp

5.1 Lỗi 1. Dùng test để chọn mô hình

Đây là lỗi nghiêm trọng vì làm kết quả test mất tính khách quan.

5.2 Lỗi 2. Đổi quá nhiều hyperparameters cùng lúc

Khi đó bạn không thể biết yếu tố nào thực sự tạo ra sự khác biệt.

5.3 Lỗi 3. Không log W&B

Khi không có log, bạn sẽ rất khó so sánh run và rất khó viết báo cáo.

5.4 Lỗi 4. Chỉ nhìn accuracy mà không nhìn loss

Accuracy không phải lúc nào cũng kể hết câu chuyện. Learning curves rất quan trọng.

5.5 Lỗi 5. Không kiểm tra dữ liệu đầu vào

Sai đường dẫn `--data_dir`, sai tên file ảnh, hoặc dữ liệu không đúng định dạng sẽ làm training lỗi ngay từ đầu.

6 Câu hỏi tự kiểm tra sau lab

Hãy tự trả lời ngắn gọn các câu hỏi sau:

1. Vì sao cần tách train / validation / test?
2. Validation dùng để làm gì?
3. Vì sao phân loại nhiều lớp dùng Cross-Entropy?
4. Khi nào AdamW có lợi hơn SGD?
5. Dấu hiệu nào cho thấy mô hình bị overfitting?
6. W&B giúp ích gì khi phải so sánh nhiều cấu hình?

Nếu bạn trả lời rõ ràng được 6 câu này, nghĩa là bạn đã hiểu đúng tinh thần của lab.